TEFAS Fon Analiz & Risk Paneli — Proje Yol Haritası

Proje Özeti

Amaç: Türkiye'deki yatırım fonlarının (Ak Portföy ve rakip şirketler) risk-getiri performansını istatistiksel yöntemlerle analiz edip, sonuçları interaktif bir web panelinde (Streamlit) sunmak.

Ekip: 2 kişi, birlikte çalışarak (pair programming / paylaşımlı görevler) **Süre:** 3-4 hafta, haftada 10-12 saat **Maliyet:** 0 TL **Diller/Araçlar:** Python, pandas, scipy, scikit-learn, statsmodels, streamlit, DuckDB/CSV, Git/GitHub

---

HAFTA 1 — Veri Altyapısı

Amaç

Analiz edilebilir, temiz bir veri seti oluşturmak. Bu hafta en kritik haftadır çünkü sonraki tüm adımlar bu veriye dayanır.

Görevler

1. Ortam kurulumu (Gün 1) - Python sanal ortamı (venv) oluşturun - `requirements.txt` dosyasını hazırlayın: `pandas`, `numpy`, `requests`, `scipy`, `scikit-learn`, `statsmodels`, `streamlit`, `duckdb`, `yfinance`, `plotly` - GitHub reposunu oluşturun, ilk commit'i atın (boş klasör yapısıyla)

2. TEFAS veri çekme (Gün 1-3) - TEFAS'ın resmi olmayan crawler paketlerinden birini araştırın ve test edin (GitHub'da birkaç seçenek var, hangisinin güncel çalıştığını kontrol edin) - Analiz edilecek 15-20 fonu belirleyin: Ak Portföy'den birkaç fon (hisse, karma, değişken) + İş Portföy, Garanti Portföy, QNB Portföy gibi rakiplerden benzer türde fonlar - Her fon için son 2-3 yıllık günlük NAV (fon birim pay değeri) verisini çekin - **Dikkat:** TEFAS endpoint'i zaman zaman değişebilir, veri çekme kodunu esnek/hataya dayanıklı yazın (try-except blokları, retry mantığı)

3. Benchmark verisi (Gün 3) - `yfinance` ile BIST100 endeksinin (`XU100.IS`) aynı tarih aralığındaki günlük verisini çekin

4. Veri temizleme (Gün 4-5) - Eksik günleri (tatil, veri hatası) tespit edip uygun şekilde işleyin (forward-fill veya çıkarma) - Sıfır/negatif/anormal değerleri kontrol edin - Tüm fonların ve benchmark'ın aynı tarih aralığında hizalandığından emin olun

5. Veriyi kaydetme (Gün 5) - Temizlenmiş veriyi DuckDB veya CSV formatında `data/processed/` klasörüne kaydedin

Bu Hafta Sonunda Elinizde Olması Gerekenler

- Çalışan bir veri çekme scripti (`src/data_collection.py`)
- Çalışan bir veri temizleme scripti (`src/data_cleaning.py`)
- 15-20 fon + BIST100 için temiz, hizalanmış günlük getiri verisi
- GitHub'da düzenli commit geçmişi

Olası Zorluklar

- TEFAS paketi çalışmayabilir → alternatif paket deneyin veya endpoint'i manuel inceleyin
- Bazı fonlarda veri eksik olabilir → o fonu listeden çıkarıp yerine başka bir fon seçebilirsiniz

---

HAFTA 2 — Risk-Getiri Metrikleri ve Benchmark Karşılaştırması

Amaç

Her fonun risk ve getiri karakteristiğini sayısal olarak ortaya koymak, BIST100'e göre nasıl performans gösterdiğini ölçmek.

Görevler

1. Temel getiri hesaplamaları (Gün 1) - Günlük getiri serilerini hesaplayın (`pct_change()`) - Kümülatif getiri, yıllıklandırılmış getiri hesaplayın

2. Risk metrikleri (Gün 2-3) - **Volatilite:** Günlük getirilerin standart sapması, yıllıklandırılmış - **Sharpe oranı:** (Getiri − risksiz faiz oranı) / volatilite (risksiz oran için TCMB gösterge faizi kullanılabilir) - **Sortino oranı:** Sadece negatif volatiliteyi (downside deviation) dikkate alan versiyon - **Maksimum düşüş (Max Drawdown):** Fonun tepe noktasından en büyük değer kaybı - **Value at Risk (VaR):** Örneğin %95 güven aralığında, belirli bir sürede beklenen maksimum kayıp

3. Benchmark karşılaştırması (Gün 4-5) - **Beta:** Fonun BIST100'e karşı hassasiyeti (basit doğrusal regresyon ile) - **Tracking error:** Fon getirisi ile benchmark getirisi arasındaki farkın volatilitesi - **Information ratio:** Aktif getiri / tracking error (fonun benchmark'ı ne kadar verimli geçtiğinin ölçüsü)

4. Streamlit iskeleti (paralel olarak, Gün 3-5) - Temel sayfa yapısını kurun: fon seçme dropdown'ı, boş grafik alanları - Metrikler hazır olduğunda buraya bağlanacak

Bu Hafta Sonunda Elinizde Olması Gerekenler

- `src/metrics.py`: Sharpe, Sortino, VaR, drawdown fonksiyonları
- `src/benchmark.py`: Beta, tracking error, information ratio fonksiyonları
- Her fon için tüm metrikleri içeren bir özet tablo (DataFrame)
- Çalışan ama henüz veri dolu olmayan bir Streamlit iskeleti

Olası Zorluklar

- Yıllıklandırma formüllerinde hata yapmak kolay (günlük veriyi doğru katsayıyla çarpmak gerekir, genelde √252) → hesaplamaları küçük bir örnek veri ile elle doğrulayın

---

HAFTA 3 — İleri Analiz: Style Analysis ve Kümeleme

Amaç

Fonların “gerçekte” nasıl davrandığını, isimlerinin ötesinde veriye dayalı olarak ortaya çıkarmak.

Görevler

1. Returns-based style analysis (Gün 1-3) - Sharpe’ın stil analizi yöntemini uygulayın: bir fonun getirisini, birkaç varlık sınıfının (örn. BIST100, tahvil endeksi, döviz, mevduat/nakit) getirilerinin ağırlıklı kombinasyonu olarak açıklamaya çalışın - Kısıtlar: ağırlıklar toplamı 1, hiçbir ağırlık negatif olamaz (`scipy.optimize.minimize` ile kısıtlı en küçük kareler) - Sonuç: her fon için “aslında %70 hisse, %20 tahvil, %10 nakit gibi davranıyor” şeklinde bir dağılım

2. Kümeleme (Gün 3-4) - Risk-getiri metriklerini (Sharpe, volatilite, max drawdown vb.) özellik olarak kullanarak K-means veya hiyerarşik kümeleme uygulayın - Kaç küme olacağına karar vermek için elbow method veya silhouette score kullanın - Sonuçları 2 boyutlu bir scatter plot ile görselleştirin (gerekirse PCA ile boyut indirgeme)

3. Dashboard entegrasyonu (Gün 4-5) - Hafta 2’deki metrik tablosunu Streamlit’e bağlayın - Style analysis sonuçlarını pasta grafiği veya yığılmış bar chart ile gösterin - Kümeleme sonuçlarını scatter plot olarak ekleyin - Fon seçimine göre grafiklerin dinamik güncellenmesini sağlayın

Bu Hafta Sonunda Elinizde Olması Gerekenler

- `src/style_analysis.py`: Kısıtlı regresyon fonksiyonu
- `src/clustering.py`: Kümeleme fonksiyonu
- Tüm analizlerin göründüğü, büyük ölçüde tamamlanmış bir dashboard

Olası Zorluklar

- Style analysis optimizasyonu bazı fonlarda iyi yakınsamayabilir → varlık sınıfı sayısını azaltıp basitleştirebilirsiniz (3-4 kategori yeterli)
- Zaman sıkışırsa: kümelemeyi basit tutun (sadece Sharpe + volatilite üzerinden 2D kümeleme yeterli olur)

HAFTA 4 — Cilalama, Test ve Sunum

Amaç

Projeyi CV’de ve mülakatta rahatça gösterilebilecek, profesyonel görünümlü bir hale getirmek.

Görevler

1. Dashboard iyileştirme (Gün 1-2) - Kullanıcı arayüzünü sadeleştirin, gereksiz karmaşıklığı azaltın - Grafik başlıkları, eksen etiketleri, renk paleti gibi detayları düzenleyin - Yükleme hızını kontrol edin (veri önbellekleme için `st.cache_data` kullanın)

2. Test (Gün 2-3) - `tests/test_metrics.py`: Sharpe, drawdown gibi hesaplamaların doğruluğunu basit örnek verilerle kontrol eden testler yazın - Uç durumları kontrol edin (eksik veri, tek günlük veri gibi)

3. Dokümantasyon (Gün 3-4) - **README.md** yazın: - Proje amacı ve motivasyon (kısa paragraf) - Kurulum adımları (`pip install -r requirements.txt`, `streamlit run app/dashboard.py`) - Kullanılan yöntemlerin kısa açıklaması - 2-3 ekran görüntüsü (dashboard'dan) - Katkıda bulunanlar bölümü - Kod içine gerekli yerlerde açıklayıcı yorum satırları ekleyin

4. Bulgular özeti (Gün 4-5) - Analiz sonuçlarından 3-5 somut çıkarım yazın. Örnek formatlar: - "İncelenen fonlar arasında en yüksek Sharpe oranına sahip olan X fonu, ama en düşük tracking error Y fonunda görüldü" - "Style analysis sonuçlarına göre Z fonu isminin aksine ağırlıklı olarak tahvil ağırlıklı davranıyor" - "Kümeleme analizi, A ve B fonlarının farklı şirketlere ait olmasına rağmen çok benzer risk profiline sahip olduğunu gösterdi"

5. Son kontrol ve GitHub'a yükleme (Gün 5) - Tüm kodun hatasız çalıştığını baştan sona test edin - Son commit'i atın, repo'yu public yapın (veya CV'de link paylaşacaksanız erişilebilir olduğundan emin olun)

Bu Hafta Sonunda Elinizde Olması Gerekenler

- Tamamlanmış, çalışan bir Streamlit dashboard'u
- Kapsamlı bir README
- Basit ama var olan testler
- GitHub'da düzenli, iki kişinin de katkısının göründüğü bir commit geçmişi

---

Proje Klasör Yapısı (Referans)

```
tefas-fon-analiz/
│
├── README.md
├── requirements.txt
├── .gitignore
│
├── data/
│   ├── raw/
│   └── processed/
│
├── src/
│   ├── data_collection.py
│   ├── data_cleaning.py
│   ├── metrics.py
│   ├── benchmark.py
│   ├── style_analysis.py
│   └── clustering.py
│
├── notebooks/
│   └── exploration.ipynb
│
├── app/
│   └── dashboard.py
│
└── tests/
    └── test_metrics.py
```

Genel Tavsiyeler

- **Her hafta sonunda mutlaka commit atın.** Son güne bırakmak hem stresli olur hem de GitHub geçmişinizin "düzenli çalışıldı"

izlenimini kaybetmesine yol açar.
- **İkiniz de her adımı anlayın.** Pair programming yapıyorsanız bile, mülakatta “bu kısmı nasıl yaptınız” sorusuna ikiniz de rahatça cevap verebilmelisiniz.
- **Zaman sıkışırsa kapsamı daraltın, kaliteyi düşürmeyin.** Örneğin fon sayısını 20’den 12’ye indirmek, style analysis’i basitleştirmek gibi esneklikler her zaman mümkün — ama yarım kalmış, hata veren bir dashboard’dan çok daha iyidir.
- **Sonuçları yorumlamayı ihmal etmeyin.** Sadece grafik üretmek yetmez; “bu grafik ne anlatıyor” sorusuna cevap veren kısa yorumlar eklemek, projenin istatistik bölümü öğrencisi kimliğinizi öne çıkarır.

Bu yol haritası esnektir — süreç içinde ihtiyaca göre haftaların içeriği güncellenebilir. Takıldığınız teknik noktalarda (özellikle veri çekme veya style analysis kısmında) yardım isteyebilirsiniz.